

BACSE202 – Database Systems

Translating SQL Queries into Relational Algebra – Complete Notes

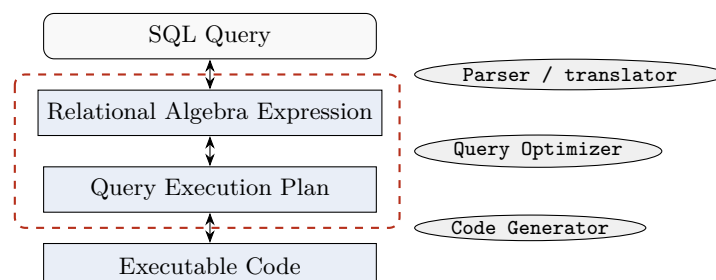
Module 3. Built from the course slides (BACSE202-DBS-Module-3, slides 187–274) and Elmasri & Navathe, Ch. 8 and 18.

Contents

1	Role of Relational Algebra in a DBMS	1
2	The Core Translation Rule	2
3	Unary Operations: σ, π, ρ	2
3.1	SELECT σ (from WHERE)	2
3.2	PROJECT π (from the SELECT list)	3
3.3	Selection + Projection together	3
3.4	RENAME ρ and the assignment arrow $\leftarrow$	3
4	Set Operations: $\cup$, $\cap$, $-$ (from UNION, INTERSECT, EXCEPT)	4
5	Cartesian Product $\times$ (from FROM R, S / CROSS JOIN)	4
6	Joins $\bowtie$ (from WHERE join conditions or JOIN ...ON)	5
6.1	Theta join	5
6.2	Equi-join	5
6.3	Natural join	5
7	Outer Joins $\bowtie\!\!\!\bowtie$, $\bowtie\!\!\!\bowtie\!\!\!$, $\bowtie\!\!\!\bowtie\!\!\!\bowtie$	6
8	Nested Queries and Query Blocks	6
9	Division $\div$ (“for ALL” queries)	7
10	Aggregate Functions and Grouping (Extended RA)	8
11	Complete Translation: Query Q_2, Query Tree and Query Graph	9
12	Step-by-Step Recipe and Practice	10
13	Errors in the Slides	11
14	Quick Revision	11

1 Role of Relational Algebra in a DBMS

SQL is *declarative* (it says *what* to fetch). Relational algebra (RA) is *procedural* (it says *how*, as a sequence of operations). The DBMS turns every SQL query into RA before running it.



- **Relational algebra:** the basic set of operations for the relational model. Each operation takes one or two relations and returns a **new relation**, so operations can be **nested**. A sequence of RA operations is a **relational algebra expression**.
- Query processing has three steps: (1) **parsing and translation** (SQL $\rightarrow$ RA), (2) **optimization**, (3) **evaluation**.

2 The Core Translation Rule

The one rule to remember

$$\underbrace{\text{SELECT } A_1, \dots, A_k}_{\pi} \underbrace{\text{FROM } R_1, \dots, R_n}_{\times} \underbrace{\text{WHERE } C}_{\sigma} \implies \pi_{A_1, \dots, A_k}(\sigma_C(R_1 \times R_2 \times \dots \times R_n))$$

Evaluate from the **inside out**: **FROM** (product) $\rightarrow$ **WHERE** (selection) $\rightarrow$ **SELECT** (projection).

SQL clause / keyword	RA operator	Note
SELECT list	π (project)	SELECT * $\Rightarrow$ no π at all
FROM R, S	$R \times S$	comma = Cartesian product
WHERE condition	σ (select)	AND $\rightarrow \wedge$, OR $\rightarrow \vee$, NOT $\rightarrow \neg$
FROM R, S WHERE R.a = S.b	$R \bowtie_{R.a=S.b} S$	σ over $\times$ = join
JOIN ... ON c / USING / NATURAL JOIN	$\bowtie_c$ / equi-join / * or $\bowtie$	
LEFT / RIGHT / FULL OUTER JOIN	$\Join$ $\Join$ $\Join$	
AS (alias, column rename)	ρ (rename)	
UNION / INTERSECT / EXCEPT	$\cup$ $\cap$ $-$	operands must be union-compatible
IN (subquery)	join / semi-join $\ltimes$	
NOT IN, NOT EXISTS	set difference $-$	
“for all / every”	division $\div$	
COUNT, SUM, AVG, MIN, MAX, GROUP BY	$\mathcal{F}$ or γ (extended RA)	HAVING $\rightarrow \sigma$ applied <i>after</i> $\mathcal{F}$
DISTINCT	(automatic)	RA is set-based: duplicates are always removed

SQL vs RA: sets and bags

RA relations are **sets**, so π removes duplicates automatically. SQL tables are **multisets (bags)**: SELECT keeps duplicates unless you write DISTINCT, and UNION ALL keeps duplicates while UNION removes them. For exam translations, treat SELECT A as π_A .

Schemas used in the examples

- **University:** Student(sid, name, age, dept), Enroll(sid, cid, grade), Course(cid, cname).
- **COMPANY** (Elmasri, slide 192; keys underlined):
 EMPLOYEE(FNAME, MINIT, LNAME, SSN, BDATE, ADDRESS, SEX, SALARY, SUPERSSN, DNO)
 DEPARTMENT(DNAME, DNUMBER, MGRSSN, MGRSTARTDATE)
 DEPT_LOCATIONS(DNUMBER, DLOCATION)
 PROJECT(PNAME, PNUMBER, PLOCATION, DNUM)
 WORKS_ON(ESSN, PNO, HOURS)
 DEPENDENT(ESSN, DEPENDENT_NAME, SEX, BDATE, RELATIONSHIP)

3 Unary Operations: σ , π , ρ

3.1 SELECT σ (from WHERE)

$\sigma_p(r)$ keeps the **tuples (rows)** of r that satisfy predicate p . Predicates use $=, \neq, <, \leq, >, \geq$ joined by $\wedge, \vee, \neg$.

- Result has the **same attributes** (degree) as r and at most as many tuples: $|\sigma_p(r)| \leq |r|$.
- **Commutative:** $\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R)) = \sigma_{c_1 \wedge c_2}(R)$ (cascade of σ).
- **Selection reduces rows, never columns.**

Examples (slides 194–196, 199)

SELECT * FROM Student WHERE dept = 'CSE';	$\sigma_{dept='CSE'}(\text{Student})$ (no π : SELECT *)
SELECT * FROM EMPLOYEE WHERE SALARY > 30000;	$\sigma_{SALARY>30000}(\text{EMPLOYEE})$
... WHERE Id > 3000 OR Hobby = 'hiking'	$\sigma_{Id>3000 \vee Hobby='hiking'}(\text{Person})$
... WHERE Id > 3000 AND Id < 3999	$\sigma_{Id>3000 \wedge Id<3999}(\text{Person})$
... WHERE NOT (Hobby = 'hiking')	$\sigma_{\neg(Hobby='hiking')}(\text{Person}) \equiv \sigma_{Hobby \neq 'hiking'}(\text{Person})$
... WHERE (DNO=4 AND SALARY>25000) OR (DNO=5 AND SALARY>30000)	$\sigma_{(DNO=4 \wedge SALARY>25000) \vee (DNO=5 \wedge SALARY>30000)}(\text{EMPLOYEE})$

Result of the last one on the COMPANY data: Franklin Wong (DNO 5, 40000), Jennifer Wallace (DNO 4, 43000), Ramesh Narayan (DNO 5, 38000).

3.2 PROJECT π (from the SELECT list)

$\pi_{A_1, \dots, A_k}(r)$ keeps only the listed **attributes (columns)** and **removes duplicate tuples**.

- $|\pi(r)| \leq |r|$; if the list contains a key, $|\pi(r)| = |r|$.
- **Not** commutative, but a cascade collapses: $\pi_{L_1}(\pi_{L_2}(R)) = \pi_{L_1}(R)$ when $L_1 \subseteq L_2$.

Example: duplicate elimination (slide 199, Fig. 7.8)

$\pi_{LNAME, FNAME, SALARY}(\text{EMPLOYEE})$ returns all 8 employees. However, $\pi_{SEX, SALARY}(\text{EMPLOYEE})$ returns only **7** rows: Zelaya and English are both $\langle F, 25000 \rangle$, so that tuple appears once. The SQL `SELECT SEX, SALARY FROM EMPLOYEE` would give 8 rows; `SELECT DISTINCT` gives 7.

3.3 Selection + Projection together

Slide 198

SELECT name, age FROM Student WHERE dept = 'CSE';

$$\pi_{name, age}(\sigma_{dept='CSE'}(\text{Student}))$$

First σ (row filter), then π (column filter). Selection must come first here: if π ran first, the attribute `dept` needed for filtering would be gone.

3.4 RENAME ρ and the assignment arrow $\leftarrow$

- $\rho_{S(B_1, \dots, B_n)}(R)$ renames relation R to S and its attributes to $B_1, \dots, B_n$. Also written $\rho_S(R)$ (relation only) or $\rho_{(B_1, \dots, B_n)}(R)$ (attributes only). The slides write $\rho_{a/b}(R)$ for “rename attribute b to a ”.
- SQL `AS` aliases map to ρ , e.g. `FROM EMPLOYEE AS E` $\Rightarrow \rho_E(\text{EMPLOYEE})$.
- A long expression can be split into named steps with $\leftarrow$ (slide 201):

In-line vs step-by-step

SELECT FNAME, LNAME, SALARY FROM EMPLOYEE WHERE DNO = 5;

$$\pi_{FNAME, LNAME, SALARY}(\sigma_{DNO=5}(\text{EMPLOYEE}))$$

$$\text{DEP5_EMPS} \leftarrow \sigma_{DNO=5}(\text{EMPLOYEE})$$

$$\text{RESULT} \leftarrow \pi_{FNAME, LNAME, SALARY}(\text{DEP5_EMPS})$$

Renaming the result's attributes: $\text{RESULT}(\text{First}, \text{Last}, \text{Salary}) \leftarrow \pi_{FNAME, LNAME, SALARY}(\text{DEP5_EMPS})$.

Multi-table SELECT-FROM-WHERE (slide 202)

SELECT movieTitle FROM StarsIn, MovieStar WHERE starName = name AND birthdate = 1960;

$$\pi_{movieTitle}(\sigma_{starName=name \wedge birthdate=1960}(\text{StarsIn} \times \text{MovieStar}))$$

This is exactly the core rule: `FROM` $\rightarrow \times$, `WHERE` $\rightarrow \sigma$, `SELECT` $\rightarrow \pi$.

4 Set Operations: $\cup$, $\cap$, $-$ (from UNION, INTERSECT, EXCEPT)

Union compatibility (required for $\cup$, $\cap$, $-$)

$R(A_1, \dots, A_n)$ and $S(B_1, \dots, B_n)$ are union-compatible if they have the **same number of attributes** and $\text{dom}(A_i) = \text{dom}(B_i)$ for every i . The result takes the attribute names of the **first** relation. This is why we usually π both sides to the same columns first.

Op	SQL	RA	Result
Union	UNION	$R \cup S$	tuples in R or S (duplicates removed). Commutative, associative.
Intersection	INTERSECT	$R \cap S$	tuples in both . Commutative, associative. Derived: $R \cap S = R - (R - S)$.
Difference	EXCEPT / MINUS	$R - S$	tuples in R but not in S . Not commutative: $R - S \neq S - R$.

SQL $\rightarrow$ RA (slides 207, 211, 215)

SELECT sid FROM Enroll UNION SELECT sid FROM Student;	$\pi_{sid}(\text{Enroll}) \cup \pi_{sid}(\text{Student})$
SELECT sid FROM Student INTERSECT SELECT sid FROM Enroll;	$\pi_{sid}(\text{Student}) \cap \pi_{sid}(\text{Enroll})$ (students enrolled in something)
SELECT sid FROM Student EXCEPT SELECT sid FROM Enroll;	$\pi_{sid}(\text{Student}) - \pi_{sid}(\text{Enroll})$ (students not enrolled in any course)

Worked tables (slides 206, 210, 214, 216)

Course_1		Course_2		Course_1 $\cup$ Course_2		Course_1 $-$ Course_2	
C_id	C_name	C_id	C_name	C_id	C_name	C_id	C_name
11	Foundation C	12	Python	11	Foundation C	11	Foundation C
21	C++	21	C++	21	C++	31	JAVA
31	JAVA			31	JAVA		
				12	Python		
						Course_1 $\cap$ Course_2	
						21	C++

A			B			A $\cap$ B		
k	x	y	k	x	y	k	x	y
1	A	2	1	A	2	1	A	2
2	B	4	4	D	8			
3	C	6	5	E	10			

$X1(\text{Name, Age}) = \{\text{Anoop 22, Saurav 22, Rakesh 20, Pritesh 19}\}$
 $X2(\text{Name, Age}) = \{\text{Anoop 22, Anurag 23, Ganesh 21, Saurav 22, Rakesh 20}\}$
 $X1 - X2 = \{\text{Pritesh 19}\}$
 $X2 - X1 = \{\text{Anurag 23, Ganesh 21}\} \Rightarrow$ difference is **not commutative**.

5 Cartesian Product $\times$ (from FROM R, S / CROSS JOIN)

$R \times S$ combines **every** tuple of R with **every** tuple of S . No union compatibility needed.

$$\text{degree}(R \times S) = n_R + n_S, \quad |R \times S| = |R| \cdot |S|$$

On its own it is rarely useful. It becomes meaningful when followed by a σ that matches related tuples.

Examples (slides 218, 220, 221)

SELECT * FROM A CROSS JOIN B; with $A = \{1, 2, 3\}$, $B = \{x, y, z\} \Rightarrow A \times B$ has $3 \times 3 = 9$ tuples: (1,x), (1,y), (1,z), (2,x), ..., (3,z). $A1(\text{Name, RollNo})$ with 2 rows $\times$ $A2(\text{Name, RollNo})$ with 3 rows $\Rightarrow$ 6 rows, 4 columns. **Step-by-step with product** (female employees and their dependents):

$\text{FEMALE_EMPS} \leftarrow \sigma_{SEX='F'}(\text{EMPLOYEE})$
 $\text{EMP_NAMES} \leftarrow \pi_{FNAME, LNAME, SSN}(\text{FEMALE_EMPS})$

$\text{EMP_DEPENDENTS} \leftarrow \text{EMP_NAMES} \times \text{DEPENDENT}$
 $\text{ACTUAL_DEPS} \leftarrow \sigma_{SSN=ESSN}(\text{EMP_DEPENDENTS})$

$\text{RESULT} \leftarrow \pi_{FNAME, LNAME, DEPENDENT_NAME}(\text{ACTUAL_DEPS})$

The last two steps ($\times$ then $\sigma_{SSN=ESSN}$) are exactly a join, $\text{EMP_NAMES} \bowtie_{SSN=ESSN} \text{DEPENDENT}$.

6 Joins $\bowtie$ (from WHERE join conditions or JOIN ...ON)

Join = product + selection

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

Whenever an SQL WHERE clause compares attributes of **two different tables**, replace “ $\times$ then σ ” by $\bowtie$.

Join	Notation	Meaning
Theta join (conditional)	$R \bowtie_{\theta} S$	θ is any comparison $A \text{ op } B$, $\text{op} \in \{=, \neq, <, \leq, >, \geq\}$
Equi-join	$R \bowtie_{A=B} S$	θ uses only $=$; both join columns stay in the result
Natural join	$R * S$ or $R \bowtie S$	equi-join on all same-named attributes , keeping one copy of each

6.1 Theta join

Customer $\bowtie_{\text{Customer.cid} > \text{Order.oid}}$ Order (slide 227)

Customer			Order		Result				
Cid	Cname	Age	Oid	Oname	Cid	Cname	Age	Oid	Oname
101	Ajay	20	101	Pizza	102	Vijay	19	101	Pizza
102	Vijay	19	101	Noodles	102	Vijay	19	101	Noodles
103	Sita	21	103	Burger	103	Sita	21	101	Pizza
					103	Sita	21	101	Noodles

Ajay (101) is not $>$ any Oid. Nobody's Cid is $>$ 103, so Burger never appears.

6.2 Equi-join

Slides 228, 229, 231, 232, 251

SELECT Student.name, Enroll.grade FROM Student, Enroll WHERE Student.sid = Enroll.sid;

$$\pi_{\text{name, grade}}(\text{Student} \bowtie_{\text{Student.sid}=\text{Enroll.sid}} \text{Enroll}) \quad (\text{i.e. } \text{Student} \times \text{Enroll} + \sigma_{\text{Student.sid}=\text{Enroll.sid}})$$

SELECT name FROM Student, Enroll WHERE Student.sid = Enroll.sid AND grade = 'A';

$$\pi_{\text{name}}(\sigma_{\text{grade}='A'}(\text{Student} \bowtie_{\text{Student.sid}=\text{Enroll.sid}} \text{Enroll}))$$

Distinguish the two kinds of WHERE conditions: **join condition** ($\text{Student.sid} = \text{Enroll.sid}$, links two tables $\rightarrow \bowtie$) and **filter condition** ($\text{grade} = 'A'$, one table $\rightarrow \sigma$).

SELECT * FROM Faculty, Offering WHERE Faculty.FacSSN = Offering.FacSSN;

$$\text{RESULT} \leftarrow \text{Faculty} \bowtie_{\text{Faculty.FacSSN}=\text{Offering.FacSSN}} \text{Offering}$$

joe (111-11-1111) matches offerings 1111 and 3333; sue matches 2222; sara matches none and is dropped. Both FacSSN columns appear in the result.

Sailors: $S1 \bowtie_{S1.sid=R1.sid} R1$ gives (22, dustin, 7, 45.0, 101, 10/10/96) and (58, rusty, 10, 35.0, 103, 11/12/96). The sid in R1 is a **foreign key** to S1.

Manager of each department: $\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie_{\text{MGRSSN}=\text{SSN}} \text{EMPLOYEE} \Rightarrow \text{Research-Wong, Administration-Wallace, Headquarters-Borg.}$

6.3 Natural join

Only possible when the relations share an attribute with the **same name and type**. The duplicate column is dropped. If there is no common attribute, $R * S = R \times S$.

Slides 235–237

SELECT * FROM CLASS NATURAL JOIN CLASSINFO; (or INNER JOIN CLASSINFO USING (ID)) $\Rightarrow$ CLASS*CLASSINFO

CLASS	
ID	NAME
1	AAA
2	BBB
4	DDD

CLASSINFO	
ID	ADDRESS
1	CHENNAI
2	MUMBAI
3	DELHI

CLASS * CLASSINFO		
ID	NAME	ADDRESS
1	AAA	CHENNAI
2	BBB	MUMBAI

Employee(*Name, EmpId, DeptName*) * Dept(*DeptName, Manager*) joins on **DeptName**: Harry–George, Sally–Harriet, George–George, Harriet–Harriet. Production (Charles) has no employee and is dropped.

$r(A, B, C, D) \bowtie s(B, D, E)$ joins on **both** *B* and *D* and gives 5 tuples: $(\alpha, 1, \alpha, a, \alpha)$, $(\alpha, 1, \alpha, a, \gamma)$, $(\alpha, 1, \gamma, a, \alpha)$, $(\alpha, 1, \gamma, a, \gamma)$, $(\delta, 2, \beta, b, \delta)$.

Textbook: $\text{PROJ_DEPT} \leftarrow \text{PROJECT} * \rho_{(DNAME, DNUM, MGRSSN, MGRSTARTDATE)}(\text{DEPARTMENT})$. The rename makes **DNUMBER** into **DNUM** so that natural join finds the common attribute.

7 Outer Joins $\bowtie$, $\bowtie$, $\bowtie$

A normal (inner) join drops tuples that have no match. An **outer join** keeps them and pads the missing side with **NULL**.

Join	RA	SQL	Keeps
Left outer	$R \bowtie S$	LEFT OUTER JOIN	all tuples of <i>R</i> (left); unmatched $\rightarrow$ NULLs for <i>S</i> 's attributes
Right outer	$R \bowtie S$	RIGHT OUTER JOIN	all tuples of <i>S</i> (right); unmatched $\rightarrow$ NULLs for <i>R</i> 's attributes
Full outer	$R \bowtie S$	FULL OUTER JOIN	all tuples of both sides

CLASS / CLASSINFO on ID (slides 241, 245, 249)

Left: CLASS $\bowtie$ CLASSINFO		
ID	NAME	ADDRESS
1	AAA	CHENNAI
2	BBB	MUMBAI
4	DDD	NULL

Right: CLASS $\bowtie$ CLASSINFO		
ID	NAME	ADDRESS
1	AAA	CHENNAI
2	BBB	MUMBAI
3	NULL	DELHI

Full: CLASS $\bowtie$ CLASSINFO		
ID	NAME	ADDRESS
1	AAA	CHENNAI
2	BBB	MUMBAI
3	NULL	DELHI
4	DDD	NULL

SQL: SELECT * FROM CLASS NATURAL LEFT | RIGHT | FULL OUTER JOIN CLASSINFO;

More slide examples (240, 242, 244, 246, 248, 250, 252)

- DEPT $\bowtie_{DEPT_ID}$ EMPLOYEE: Account has James and Rose, Design has Kathy and Marry, and Testing (30) has **no employee**, so it is kept with NULLs.
- EMPLOYEE $\bowtie_{DEPT_ID}$ DEPT: the same 4 matches, plus the row (NULL, NULL, NULL, 30, Testing).
- EMPLOYEE $\bowtie_{DEPT_ID}$ DEPT, where Alex (105) has DEPT_ID 40 (no such department): the 4 matches, plus (105, Alex, 40, NULL, NULL), plus (NULL, NULL, NULL, 30, Testing).
- Table1(Products, Price) $\bowtie$ Table2(Products, Quantity): Kiwis 10, Onions 6, Tomatoes $\rightarrow$ Quantity NULL. Broccoli (right only) is dropped.
- Customers $\bowtie_{CustomerId}$ Orders: Shree–100, Basavaraj–300, Kalpana–NULL order, NULL customer–order 200 (CustomerId 4 does not exist).
- PARTS $\bowtie_{PROD\#}$ PRODUCTS keeps OIL (160) with NULL price. $\bowtie$ keeps product 505 (3.70) with NULL part. $\bowtie$ keeps both.

8 Nested Queries and Query Blocks

Query block (slide 270)

A **query block** is the basic unit that is translated into RA and optimized. It contains a **single** SELECT-FROM-WHERE expression, plus GROUP BY and HAVING if they are part of that block.

- Nested queries** inside a query are identified as **separate query blocks** and translated separately.
- Aggregate operators** (MAX, COUNT, ...) must be included in the **extended algebra** ($\mathcal{F}$).

Scalar subquery (slide 271)

```
SELECT LNAME, FNAME
FROM   EMPLOYEE
WHERE  SALARY > ( SELECT MAX(SALARY) FROM EMPLOYEE WHERE DNO = 5 );
```

The query is split into **two blocks**:

Inner block	Outer block
<code>SELECT MAX(SALARY) FROM EMPLOYEE WHERE DNO = 5</code> $c \leftarrow \mathcal{F}_{MAX\ SALARY}(\sigma_{DNO=5}(EMPLOYEE))$	<code>SELECT LNAME, FNAME FROM EMPLOYEE WHERE SALARY > c</code> $\pi_{LNAME,FNAME}(\sigma_{SALARY>c}(EMPLOYEE))$
The inner block is uncorrelated: it is evaluated once and its result c (a constant) is used in the outer block. On the COMPANY data, $c = 40000$ (Wong) and the answer is Wallace (43000) and Borg (55000).	

IN $\rightarrow$ join (slide 253)

`SELECT name FROM Student WHERE sid IN (SELECT sid FROM Enroll);`
 Step 1 (inner block): $\pi_{sid}(Enroll)$. Step 2 (equivalent using a join):

$$\begin{aligned} &\pi_{name}(Student \bowtie Enroll) \\ &\text{or exactly} \\ &\pi_{name}(Student \ltimes Enroll) \end{aligned}$$

IN is usually convertible into a **semi-join** $\ltimes$ (keeps the Student tuples that have at least one match, without adding Enroll's columns).

NOT IN $\rightarrow$ difference (slide 254, corrected)

`SELECT name FROM Student WHERE sid NOT IN (SELECT sid FROM Enroll);`

$$\pi_{name}\left(Student \bowtie (\pi_{sid}(Student) - \pi_{sid}(Enroll))\right)$$

Find the **sids** that are not enrolled (a difference of two union-compatible π_{sid} relations), then join back to Student to get their names. An equivalent form is $\pi_{name}(Student - \pi_{sid,name,age,dept}(Student \bowtie Enroll))$. NOT EXISTS translates the same way.

9 Division $\div$ (“for ALL” queries)

Definition

For $R(A, B)$ and $S(B)$, $R \div S$ returns the A -values that are paired in R with **every** B -value in S :

$$R \div S = \{t[A] \mid t \in R \wedge \forall s \in S \langle t[A], s \rangle \in R\}$$

In terms of basic operators:

$$\begin{aligned} T_1 &\leftarrow \pi_A(R), \\ T_2 &\leftarrow \pi_A((T_1 \times S) - R), \\ R \div S &= T_1 - T_2 \end{aligned}$$

$T_1 \times S$ is “every candidate paired with every required value”. Subtracting R leaves the missing pairs, so T_2 holds the candidates missing at least one value.

Use division when the question says **all**, **every** or **for all**: “persons with an account in *all* banks of a city”, “students who have taken *all* courses required to graduate”.

Slide 256: $A \div B1, A \div B2, A \div B3$

	Divisor	Result	Why
$A(x, y) = \{(x1,y1), (x1,y2), (x1,y3), (x1,y4), (x2,y1), (x2,y2), (x3,y2), (x4,y2), (x4,y4)\}$	$B1 = \{y2\}$	$\{x1, x2, x3, x4\}$	every x has $y2$
	$B2 = \{y2, y4\}$	$\{x1, x4\}$	only $x1$ and $x4$ have both $y2$ and $y4$
	$B3 = \{y1, y2, y4\}$	$\{x1\}$	only $x1$ has all three
Check $A \div B2$ with the formula: $T_1 = \{x1, x2, x3, x4\}$. $T_1 \times B2$ has 8 pairs. The pairs not in A are $(x2,y4)$ and $(x3,y4)$, so $T_2 = \{x2, x3\}$ and $T_1 - T_2 = \{x1, x4\}$.			

Students enrolled in ALL courses (slide 257, SQL corrected)

```
SELECT sid FROM Enroll
GROUP BY sid
HAVING COUNT(DISTINCT cid) = (SELECT COUNT(*) FROM Course);
```

$$\pi_{sid, cid}(Enroll) \div \pi_{cid}(Course)$$

Note: $\pi_{sid, cid}$ is needed first. Without it, **grade** would become part of the “A” side and break the division.
Textbook (COMPANY): employees who work on *all* projects controlled by department 5:

```
DEPT5_PROJS ← ρ(PNO)(π_PNUMBER(σ_DNUM=5(PROJECT)))
EMP_PROJ ← ρ(SSN, PNO)(π_ESSN, PNO(WORKS_ON))

RESULT_SSNS ← EMP_PROJ ÷ DEPT5_PROJS
RESULT ← π_LNAME, FNAME(RESULT_SSNS * EMPLOYEE)
```

10 Aggregate Functions and Grouping (Extended RA)

Classical RA has **no** aggregation. The **extended** RA adds the aggregate function operator. Elmasri writes it $\mathcal{F}$ (script F), and many other texts use γ :

$$\langle \text{grouping attributes} \rangle \mathcal{F} \langle \text{function list} \rangle (R)$$

or

$$\gamma \langle \text{grouping attrs} \rangle, \langle \text{function list} \rangle (R)$$

Functions: SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT. The attributes **before** $\mathcal{F}$ are the GROUP BY columns. If there are none, the whole relation is one group and the result is a single tuple.

SQL → extended RA (slides 259, 263)

SELECT COUNT(*) FROM Student;	$\gamma_{count(*)}(Student)$ or $\mathcal{F}_{COUNT \ sid}(Student)$
SELECT dept, COUNT(*) FROM Student GROUP BY dept;	$\gamma_{dept, count(*)}(Student)$ or $dept \mathcal{F}_{COUNT \ sid}(Student)$
SELECT MAX(Salary) FROM Employee;	$\mathcal{F}_{MAX \ Salary}(Employee)$
SELECT MIN(Salary) FROM Employee;	$\mathcal{F}_{MIN \ Salary}(Employee)$
SELECT SUM(Salary) FROM Employee;	$\mathcal{F}_{SUM \ Salary}(Employee)$

GROUP BY + HAVING, and aggregates with a join

```
SELECT DNO, COUNT(*), AVG(SALARY) FROM EMPLOYEE GROUP BY DNO HAVING AVG(SALARY) > 30000;
```

$$T \leftarrow \rho_{(DNO, NO_EMPS, AVG_SAL)} \left(DNO \mathcal{F}_{COUNT \ SSN, AVERAGE \ SALARY}(EMPLOYEE) \right)$$

$$RESULT \leftarrow \sigma_{AVG_SAL > 30000}(T)$$

WHERE becomes a σ **below** $\mathcal{F}$ (it filters rows before grouping). HAVING becomes a σ **above** $\mathcal{F}$ (it filters groups after grouping). On the COMPANY data the groups are DNO 5 (4 employees, average 33250), DNO 4 (3, 31000) and DNO 1 (1, 55000). All three have an average above 30000.

Slide 260: SELECT country.country_name, COUNT(city.lat), SUM(city.lat), AVG(city.lat), MIN(city.lat), MAX(city.lat) FROM city INNER JOIN country ON city.country_id = country.id GROUP BY country.id, country.country_name;

$$id, country_name \mathcal{F}_{COUNT \ lat, SUM \ lat, AVERAGE \ lat, MIN \ lat, MAX \ lat} (city \bowtie_{city.country_id=country.id} country)$$

followed by π to drop id. USA has 2 cities, so its COUNT is 2 and its AVG (37.39) is SUM/2.

11 Complete Translation: Query Q2, Query Tree and Query Graph

Q2 (slide 272)

For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address and birth date.

```
SELECT P.PNUMBER, P.DNUM, E.LNAME, E.ADDRESS, E.BDATE
FROM   PROJECT AS P, DEPARTMENT AS D, EMPLOYEE AS E
WHERE  P.DNUM = D.DNUMBER AND D.MGRSSN = E.SSN AND P.PLOCATION = 'Stafford';
```

Step 1: canonical (direct) translation using the core rule:

$$\pi_L(\sigma_C((P \times D) \times E)), \text{ where}$$

$$L = P.PNUMBER, P.DNUM, E.LNAME, E.ADDRESS, E.BDATE$$

$$C = P.DNUM = D.DNUMBER \wedge D.MGRSSN = E.SSN \wedge P.PLOCATION = 'Stafford'$$

Step 2: sort the conditions. $P.PLOCATION = 'Stafford'$ involves one table, so it is a filter (σ). $P.DNUM = D.DNUMBER$ and $D.MGRSSN = E.SSN$ each link two tables, so they are joins ($\bowtie$).

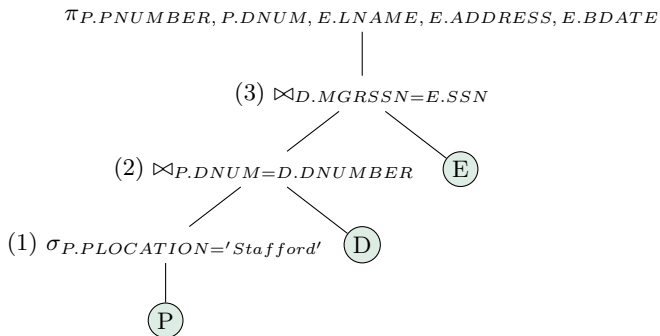
Step 3: better RA expression (textbook form):

$$\pi_{PNUMBER,DNUM,LNAME,ADDRESS,BDATE} \left(\left((\sigma_{PLOCATION='Stafford'}(PROJECT)) \right. \right. \\ \left. \left. \bowtie_{DNUM=DNUMBER} DEPARTMENT \right) \bowtie_{MGRSSN=SSN} EMPLOYEE \right)$$

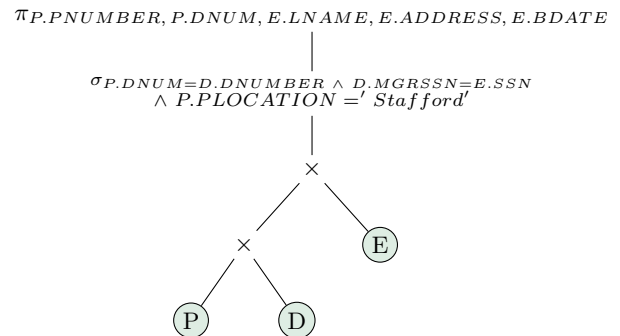
Query tree vs query graph (slide 269)

- **Query tree:** a tree for an **RA expression**. **Leaves** are the input relations, and **internal nodes** are RA operations. Execution runs an internal node as soon as its operands are available, then replaces the node by the resulting relation. The root gives the answer. One query can have **many** query trees.
- **Query graph:** a graph for a **relational calculus** expression. Relation nodes are single circles and constant nodes are double circles. Selection and join conditions are edges, and the attributes to retrieve are shown in [] above each relation. It does **not** say which operation runs first, and there is only **one** graph per query.

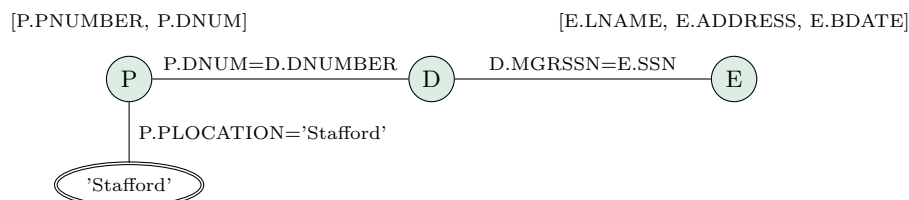
(a) Query tree for the Step-3 expression



(b) Initial (canonical) query tree



(c) Query graph for Q2



The **canonical tree** (b) is what the parser produces directly from SQL: $\times$ of the FROM tables, one big σ , then π . It is correct but very expensive, because $P \times D \times E$ is huge. Heuristic optimization (the next topic) rewrites it into trees like (a): push σ down, turn σ over $\times$ into $\bowtie$, and push π down.

Another canonical translation (slide 276)

```
SELECT LNAME FROM EMPLOYEE, WORKS_ON, PROJECT WHERE PNAME = 'Aquarius' AND PNUMBER = PNO AND
```

ESSN = SSN AND BDATE > '1957-12-31';

Canonical: $\pi_{LNAME}(\sigma_C(\text{EMPLOYEE} \times \text{WORKS_ON} \times \text{PROJECT}))$,
 $C = PNAME = 'Aquarius' \wedge PNUMBER = PNO \wedge ESSN = SSN \wedge BDATE > '1957-12-31'$

With joins: $\pi_{LNAME}((\sigma_{PNAME='Aquarius'}(\text{PROJECT}) \bowtie_{PNUMBER=PNO} \text{WORKS_ON})$
 $\bowtie_{ESSN=SSN} \sigma_{BDATE>'1957-12-31'}(\text{EMPLOYEE}))$

12 Step-by-Step Recipe and Practice

How to translate any SQL query

1. **Split into query blocks.** Translate each nested subquery separately, innermost first.
2. **FROM:** write the tables (use ρ for aliases), combined with $\times$.
3. **WHERE:** sort each condition. If it links two tables, it is a **join condition** ($\bowtie$). If it involves one table, it is a **filter** (σ , applied to that table as early as possible).
4. **Subquery predicates:** IN/EXISTS $\rightarrow$ join or semi-join. NOT IN/NOT EXISTS $\rightarrow$ difference. “All” $\rightarrow$ division. A scalar subquery becomes a constant c computed by $\mathcal{F}$.
5. **GROUP BY / aggregates:** $\mathcal{GF}_F$. **HAVING** is a σ on top of it.
6. **SELECT list:** π at the top. Skip it for **SELECT ***. Use ρ for AS column names.
7. **UNION / INTERSECT / EXCEPT:** translate each side, make them union-compatible with π , and combine with $\cup$, $\cap$ or $-$.

Practice (COMPANY schema), with answers

1. Names of employees in the ‘Research’ department.

SELECT FNAME, LNAME, ADDRESS FROM EMPLOYEE, DEPARTMENT WHERE DNAME = 'Research' AND DNUMBER = DNO;

$\pi_{FNAME,LNAME,ADDRESS}(\sigma_{DNAME='Research'}(\text{DEPARTMENT}) \bowtie_{DNUMBER=DNO} \text{EMPLOYEE})$

2. Employees who work on project ‘ProductX’.

SELECT E.FNAME, E.LNAME FROM EMPLOYEE E, WORKS_ON W, PROJECT P WHERE E.SSN = W.ESSN AND W.PNO = P.PNUMBER AND P.PNAME = 'ProductX';

$\pi_{FNAME,LNAME}((\sigma_{PNAME='ProductX'}(\text{PROJECT}) \bowtie_{PNUMBER=PNO} \text{WORKS_ON})$
 $\bowtie_{ESSN=SSN} \text{EMPLOYEE})$

3. Employees with *no* dependents.

SELECT FNAME, LNAME FROM EMPLOYEE WHERE SSN NOT IN (SELECT ESSN FROM DEPENDENT);

$\text{NO_DEP} \leftarrow \pi_{SSN}(\text{EMPLOYEE}) - \rho_{(SSN)}(\pi_{ESSN}(\text{DEPENDENT}))$
 $\text{RESULT} \leftarrow \pi_{FNAME,LNAME}(\text{NO_DEP} * \text{EMPLOYEE})$

4. Project numbers that involve an employee named ‘Smith’, either as a worker or as the department manager. (SELECT PNO FROM WORKS_ON, EMPLOYEE WHERE ESSN = SSN AND LNAME = 'Smith') UNION (SELECT PNUMBER FROM PROJECT, DEPARTMENT, EMPLOYEE WHERE DNUM = DNUMBER AND MGRSSN = SSN AND LNAME = 'Smith');

$\pi_{PNO}(\text{WORKS_ON} \bowtie_{ESSN=SSN} \sigma_{LNAME='Smith'}(\text{EMPLOYEE}))$
 $\cup \pi_{PNUMBER}(\text{PROJECT} \bowtie_{DNUM=DNUMBER}$
 $(\text{DEPARTMENT} \bowtie_{MGRSSN=SSN} \sigma_{LNAME='Smith'}(\text{EMPLOYEE})))$

5. Number of employees and average salary per department.

SELECT DNO, COUNT(*), AVG(SALARY) FROM EMPLOYEE GROUP BY DNO;

$\text{DNO } \mathcal{F}_{COUNT\ SSN, AVERAGE\ SALARY}(\text{EMPLOYEE})$

6. Every employee with their supervisor's name (self-join with rename).

SELECT E.LNAME, S.LNAME FROM EMPLOYEE AS E, EMPLOYEE AS S WHERE E.SUPERSSN = S.SSN;

$\pi_{E.LNAME, S.LNAME}(\rho_E(EMPLOYEE) \bowtie_{E.SUPERSSN=S.SSN} \rho_S(EMPLOYEE))$

13 Errors in the Slides

Corrections

Slide	Issue → correct version
194	“ <i>SALARY</i> > 30,000 (EMPLOYEE)” is missing the σ , and the output text says ‘300000’. Correct: $\sigma_{SALARY>30000}(EMPLOYEE)$.
197, 201, 219	The π and σ symbols and the $\leftarrow$ arrows are lost in the slide text. Correct forms: $\pi_{LNAME, FNAME, SALARY}(EMPLOYEE)$ and $DEP5_EMPS \leftarrow \sigma_{DNO=5}(EMPLOYEE)$.
212	The result of EMP_TEST INTERSECT EMP_DESIGN is labelled “UNION”. It is the intersection (only 104 Kathy).
228	The condition <i>Student.sid</i> = <i>Enroll.sid</i> is labelled a theta join. It is correct but more precisely an equi-join (a theta join that uses only =).
231	“Faculty.FacSSM” → Faculty. FacSSN .
249	The full outer join result is missing the ID column values and labels CLASSINFO’s second column NAME. Correct result: (1, AAA, CHENNAI), (2, BBB, MUMBAI), (3, NULL, DELHI), (4, DDD, NULL).
250	The OrderDate values in the result table do not match the Orders input table. Only the NULL pattern is meaningful.
253	$\pi_{name}(Student \bowtie Enroll)$ matches IN as a set. The exact operator is the semi-join $Student \ltimes Enroll$.
254	$Student - (Student \bowtie Enroll)$ is invalid : the two sides are not union-compatible (the join has extra attributes cid and grade). Correct: $\pi_{name}(Student \bowtie (\pi_{sid}(Student) - \pi_{sid}(Enroll)))$.
257	HAVING COUNT(cid) should be COUNT(DISTINCT cid) for the SQL to match ÷. It also assumes every cid in Enroll exists in Course.
272	“P.NUMBER” → P. PNUMBER ; “ STAFFORD ” has stray spaces. The RA line on the slide has lost its π , σ and $\bowtie$ symbols (see Q2 above).
276	“PNMUBER” → PNUMBER .

14 Quick Revision

Symbols (slide 264) and the SQL they come from

σ (sigma)	SELECT (selection)	WHERE (rows)
π (pi)	PROJECT	SELECT list (columns, duplicates removed)
ρ (rho)	RENAME	AS
$\times$ (times)	CARTESIAN PRODUCT	FROM R, S, CROSS JOIN
$\bowtie$ (bow-tie)	JOIN (theta / equi / natural *)	WHERE R.a = S.b, JOIN ...ON, NATURAL JOIN
$\bowtie \leftarrow \bowtie$	OUTER JOINS	LEFT / RIGHT / FULL OUTER JOIN
$\cup$ (cup) $\cap$ (cap) $-$ (minus)	UNION, INTERSECTION, DIFFERENCE	UNION, INTERSECT, EXCEPT
$\div$	DIVISION	“for all” / NOT EXISTS ...NOT EXISTS / COUNT = COUNT
$\mathcal{F}$ or γ	AGGREGATE (extended RA)	COUNT/SUM/AVG/MIN/MAX, GROUP BY

- **Operator classes (slide 262).** **Basic unary:** σ, π, ρ . **Basic binary:** $\cup, \times, -$. **Derived:** $\bowtie (= \sigma(\times)), \cap (= R - (R - S)), \div$ (from $\pi, \times, -$). The complete set is $\{\sigma, \pi, \cup, \rho, -, \times\}$.
- **Core rule:** $\pi_{SELECT}(\sigma_{WHERE}(\times_{FROM}))$ is the **canonical** form; turning σ over $\times$ into $\bowtie$ is the first improvement.
- **Selection before projection** whenever π would drop an attribute that σ needs.
- $|R \times S| = |R| \cdot |S|$; $\cup, \cap, -$ need union compatibility; $-$ is not commutative.
- Natural join: common attributes with the same name, one copy kept. With no common attribute, it equals $\times$.
- **Query block** = one SELECT-FROM-WHERE (+ GROUP BY/HAVING). Nested queries are separate blocks. Aggregates need extended RA.
- **WHERE** → σ below $\mathcal{F}$; **HAVING** → σ above $\mathcal{F}$.
- **Query tree:** leaves = relations, internal nodes = operators; many per query. **Query graph:** for relational calculus, no execution order, one per query.